STRESSFORGE

Design decisions: argon2, Zod and HTTP 422

Why the project differs from the course guide's suggested tools in three places, with the reasons, where each is in the code, and short answers to the questions an examiner is likely to ask. Prepared for the project presentation. CSC4161, Section D.

1. Summary

Topic	The guide suggests	STRESSFORGE uses	In one sentence
Password hashing	bcrypt	**argon2** (Argon2id)	A newer, memory-hard hash that is the current recommendation and has no 72-byte input limit.
Input validation	class-validator with DTO classes and a ValidationPipe	**Zod** with a global `ZodValidationPipe`	One strict schema is the check, the TypeScript type and the API documentation.
Validation error status	400 Bad Request	**422** Unprocessable Content, code `VALIDATION_FAILED`	Matches the HTTP standard's meaning of 422 and gives one code for every validation failure.

> The guide says equivalent tools are acceptable if the instructor accepts them. The behaviour the guide asks for is present in each case: slow salted password hashes, validation with a message naming the failed field, and a 4xx status. **Add here that your instructor accepted this, and when.**

2. Argon2 instead of bcrypt

1. **It is the current recommendation for password hashing.** Argon2 won the Password Hashing Competition in 2015 and is specified in RFC 9106. OWASP's password storage guidance lists Argon2id as its first choice.
2. **It is memory-hard.** Each hash needs a configurable amount of memory, which makes attacks with GPUs and custom hardware much more expensive. bcrypt uses very little memory, so that kind of attack is cheaper.
3. **It has no 72-byte limit.** bcrypt only uses the first 72 bytes of a password. Argon2 hashes the whole input. The project's own rule allows passwords up to 128 characters, so this matters.
4. **The cost can be tuned.** Time, memory and parallelism are separate settings, so the cost can be raised as hardware improves without changing the algorithm.
5. **It is the same idea the guide wants.** The guide's real requirement is slow, salted hashes that resist brute force. Argon2 meets that, and the salt is generated and stored inside the hash string.
6. **The usage is safe.** The code uses `argon2.hash` and `argon2.verify` from the `argon2` library, and runs a dummy hash for unknown emails so the response time does not reveal which accounts exist.

What the code actually does (checked)

Item	Fact	Where
Variant and cost	`argon2.hash(password)` is called with no options, so the library defaults apply: **Argon2id, 3 passes (timeCost), 64 MiB of memory (memoryCost 65536 KiB), 4 lanes (parallelism)**	`src/modules/auth/auth.service.ts`; defaults in `node_modules/argon2/argon2.cjs`
Version	`argon2` 0.44.0	`package.json`
Timing safety	A login for an unknown email still verifies against a precomputed dummy hash; a suspended account is reported only after the password is correct	`auth.service.ts`, `DUMMY_HASH_PROMISE` and `login()`
Password rule	8 to 128 characters with at least one letter and one digit	`RegisterDto`, `newPasswordSchema`

Item	Fact	Where
Never exposed	The hash is never selected into a response	`UsersService.toDto`

3. Zod instead of class-validator

1. **One schema gives you the check and the type.** A Zod schema is the runtime validation and the TypeScript type at once. With class-validator you write a class with decorators, and the runtime and compile-time types can drift apart.
2. **Unknown fields are rejected, not ignored.** `z.strictObject` returns an error for any key you did not list. The guide wants unknown fields stripped; rejecting them is stricter, and it is why Figure 6.7 shows `Unrecognized key`.
3. **It validates more than request bodies.** The same schemas check query strings, engine inputs and results, stored documents read back from the database (jsonb), and configuration at start-up.
4. **The contract is the single source of truth.** `nestjs-zod` turns the schemas into the Swagger documentation, so the docs cannot disagree with the code. The OpenAPI documents generated from them produce the TypeScript types the web client uses.
5. **Rules that decorators handle awkwardly are easy.** Examples: control-character filtering, the allow-listed report context, the "nothing to change" refinement, and cross-field checks.
6. **It is still a global pipe.** `ZodValidationPipe` is registered globally in `app.module.ts`, the same place the guide puts `ValidationPipe`.

What the code actually does (checked)

Item	Fact	Where
Global pipe	`{ provide: APP_PIPE, useClass: ZodValidationPipe }`	`src/app.module.ts`
Strict request bodies	`z.strictObject({ ... })` with minimum and maximum lengths, for example the report: title 3 to 120, body 10 to 4,000, no control characters, an allow-listed context	`src/common/contracts/phase3/support.schema.ts`
Configuration	The environment is parsed with Zod at start-up; the server refuses to start on a bad value	`src/config/app.config.ts`
Documentation	`cleanupOpenApiDoc` from nestjs-zod builds the Swagger document	`src/app.setup.ts`
Evidence	Figure 6.7: an empty name and an unknown field give 422 with each field named	Report, Chapter 6

4. HTTP 422 instead of 400

1. **It matches the HTTP standard.** RFC 9110 defines 422 Unprocessable Content as: the server understands the request and its syntax is correct, but it cannot process the instructions in it. A body that is valid JSON with a wrong field is exactly that.
2. **It separates two different failures.** Under that reading, 400 is for a request that is malformed, and 422 is for one that is well-formed but semantically invalid. Clients and developers can tell the two apart.
3. **It gives one code for every validation problem.** The client handles `VALIDATION_FAILED` with a single branch, and the response always has the same shape, with each failing field named in `details.fields`.
4. **It is documented and tested.** The code is in the OpenAPI contract, in the error table (Section 5.1 of the report), and in the end-to-end tests. Figure 6.7 shows it live.

What the code actually does (checked)

Item	Fact	Where
Error codes	`VALIDATION_FAILED` maps to HTTP 422; the nine codes are VALIDATION_FAILED, UNAUTHENTICATED, SESSION_EXPIRED, FORBIDDEN, NOT_FOUND, CONFLICT, RATE_LIMITED, INTERNAL and UNAVAILABLE	`src/common/errors/app.exception.ts`
Response shape	`{ "error": { "code": "VALIDATION_FAILED", "message": "Validation failed", "details": { "fields": [ { "path", "message" } ] } } }`	`src/common/filters/all-exceptions.filter.ts`
Other statuses kept distinct	401 not signed in, 403 no permission, 404 not found, 409 conflict (stale revision, duplicate report), 429 rate limit	Report, Section 5.1

> **An honest limit to state yourself.** An unparsable JSON body (broken syntax) also returns 422 in this API, because every body problem goes through the same filter. Strictly, RFC 9110 would call that malformed and use 400. The reason is the single validation code described above; say it first, before an examiner asks.

5. Likely questions and short answers

Question	Short answer
The guide says bcrypt. Why not?	The guide's requirement is slow, salted hashes that resist brute force. Argon2id meets it and is what OWASP recommends first; bcrypt also truncates input at 72 bytes and uses little memory. Nothing else in the project depends on bcrypt hashes, so a new project starts with the stronger choice.
Is argon2 allowed?	The guide allows equivalent tools with the instructor's acceptance. **State the acceptance.**
How strong is your argon2 setting?	The library defaults: Argon2id, 3 passes, 64 MiB memory, 4 lanes. I did not tune it, and it could be raised without changing the algorithm.
Why Zod and not class-validator?	One strict schema gives validation, the TypeScript type and the Swagger document, so they cannot drift. It also validates things that are not DTO classes: query strings, stored JSON, environment variables, engine results.
The guide says unknown fields are stripped. Yours rejects them?	Yes, deliberately: `strictObject` returns 422 naming the unknown key. That is stricter; nothing extra can be smuggled in, and the client learns about a typo instead of having it silently ignored.
Why 422, not 400?	RFC 9110: 422 is for a well-formed request the server cannot process. A valid JSON body with a wrong field is that. 400 stays for malformed requests, and every validation failure has one code and one shape.
Your malformed JSON also gives 422.	True. I used one code for every body problem so the client has one branch. By the strict reading of the RFC, malformed syntax is 400; I accepted that small stretch for consistency and documented it.
Did you test this?	Yes: Figure 6.7 shows the 422 live; the end-to-end suite asserts 422 for invalid and unknown fields; the unit tests cover the configuration schema.

6. Where to find it

What	Where
Report, deviations and coverage table	`docs/report/StressForge-Project-Report.docx`, Section 5.3
Report, error table with 422	Section 5.1
Report, code excerpts	Chapter 7 (7.1 strict contract, 7.4 timing-safe login, 7.5 exception filter)
Live evidence	Figure 6.7 (422), Figure 6.8 (409), Figure 6.5 (403), Figure 6.4 (401)